NEW WORLD EMERALD PRIVATE SCHOOL

Website and Portal Architecture

How the system is built, run and handed over

Live site: https://newworldemeraldprivateschool.com
Repository: github.com/ojemenosedestiny-png/new-world-emerald-school
Version: 24 October 2026 (commit 890f160 and later)
Audience: school leadership and the developer who takes over

Contains no passwords or keys. Keys live only in Render → Environment.

CONTENTS

What this document covers

#	Section
1	The system at a glance
2	Architecture diagram
3	Technology
4	How the code is organised
5	How a request works
6	Data: what is stored
7	Sign-in, roles and permissions
8	Features by area
9	Files, video and email
10	Security
11	Hosting, domain and deployment
12	Settings (environment variables)
13	Testing and quality
14	How it was developed
15	Handover checklist
16	Known items and recommended next steps

1 · OVERVIEW

The system at a glance

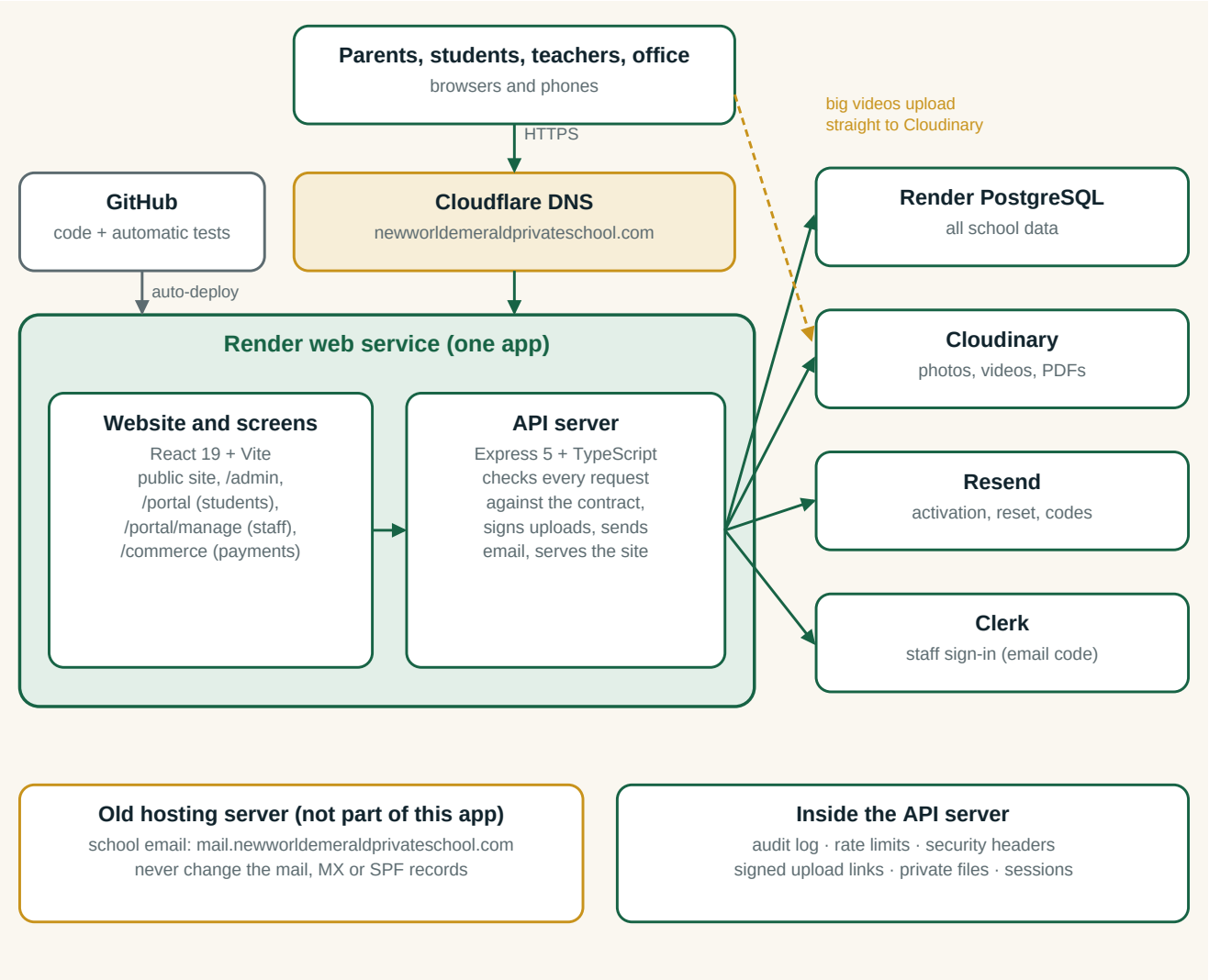
The school's website and portals are **one web application**, hosted on Render and reached at **newworldemeraldprivateschool.com**. It replaced the earlier WordPress site on 24 October 2026. Everyone uses the same address; what they see depends on who they are.

Area	Address	Who uses it	What it does
Public website	/	Families, visitors	School information, admissions form, news, live video updates, calendar, gallery, prospectus.
Payments and store	/commerce	Parents	Uniform and book orders, fee prices, school bank accounts, payment reports with screenshots.
School office admin	/admin	Website administrators	Dashboard, admissions, website content editor, live updates, calendar, store and fees, media and email checks.
Learning portal (staff)	/portal/manage	Administrators, teachers	Classes, students, teachers, exams (CBT), marking, report cards, events, gallery, photo approvals, security.
Student portal	/portal	Students	Phone-style home, exams, grade sheet, report cards, events calendar, staff directory, gallery, library, profile photo.

In one sentence: a React website and an Express API run together as one Render service, keep data in a PostgreSQL database, store files on Cloudinary, send email through Resend, and use Clerk for staff sign-in. Students sign in with their own Student ID and password, managed by the site itself.

2 · ARCHITECTURE

Architecture diagram



- **One service:** the API server also serves the built website, so sign-in cookies stay on one address and there is no separate frontend host.
- **Cloudflare** only answers “where is this domain” (DNS) and passes traffic to Render; Render provides the HTTPS certificate.
- **Outside services** are called only by the API server, except big video uploads, which go from the admin's browser straight to Cloudinary using a short-lived signed permission from the server.
- **School email** is not part of this system. It still runs on the old hosting server and must keep working independently.

3 · TECHNOLOGY

Technology

Layer	Technology	Why
Screens	React 19, Vite 7, Tailwind CSS 4, TanStack Query, wouter, framer-motion, Recharts	Fast, modern, mobile-first; pages load only the code they need.
API	Node.js 24, Express 5, TypeScript 5.9	One language across the whole project.
Contract	OpenAPI 3.1 YAML → Orval code generation	Every request and response is defined once; validators and screen hooks are generated from it.
Validation	Zod (generated)	The server rejects any request that does not match the contract.
Database	PostgreSQL 16 on Render, Drizzle ORM	Reliable relational data; table changes are applied by the build.
Staff sign-in	Clerk (production instance, free plan)	Managed accounts and email codes for staff.
Student sign-in	Built in: scrypt password hashes, server sessions	Students have no email-based accounts; the school controls them fully.
Files	Cloudinary (public CDN + private authenticated files)	Photos, videos and PDFs, with a fast CDN for the website.
Email	Resend (verified sending domain)	Activation links, confirmations, password resets, staff sign-in codes.
Hosting	Render (web service + PostgreSQL), Cloudflare DNS	Simple deploys straight from GitHub.
Quality	Node test runner, Vitest + Testing Library, GitHub Actions CI, Playwright checks	Every change is tested before and after it reaches the live site.

4 · CODEBASE

How the code is organised

The repository is a **pnpm monorepo**: several packages in one repository that share code and settings.

Folder	Contents
artifacts/new-world-emerald	The website and every screen. *src/pages* (one file per address), *src/components* (website sections, admin tools), *src/portal* (student and staff portals), *tests* (screen tests).
artifacts/api-server	The API and web server. *src/routes* (one file per area: portal, storage, commerce, accounts…), *src/lib* (security, email, Cloudinary, sign-in, tests).
lib/api-spec	The API contract (*openapi.yaml* plus *portal, reports, security, accounts, community, media* YAML files).
lib/api-zod, lib/api-client-react	Generated from the contract. Never edited by hand.
lib/db	Database tables (*src/schema*), one file per area.
scripts/render-build.sh	Render's build: checks key formats, installs, updates database tables, builds.
docs	*DEPLOY-RENDER.md* (hosting, domain, accounts, step by step) and this document.
.github/workflows/ci.yml	Automatic tests on every push to main, portal-upgrade and the deploy branch.
README.md	Starting point for developers: commands and rules.

5 · REQUEST FLOW

How a request works

Every feature follows the same path, which keeps the code predictable and safe:

- **1. Contract first.** The developer describes the new request in *lib/api-spec* (fields, limits, responses).
- **2. Generate.** *pnpm --filter @workspace/api-spec run codegen* creates Zod validators for the server and typed React Query hooks for the screens.
- **3. Server.** The route validates the request with the generated validator, checks who is asking, then reads or writes the database.
- **4. Screen.** The page calls the generated hook; data is cached and refreshed automatically.

Who is asking is decided once per request:

- **Website management** (admin tools) is **private by default**: any address not on a short public list in *website-admin.ts* requires an approved administrator (“fail closed”).
- **Portal requests** work out the role in order: administrator, then teacher, then student (from the portal cookie), otherwise guest.
- **Changes by staff** are written to the audit log automatically, with secrets removed.

6 · DATA

Data: what is stored

All data lives in one PostgreSQL database on Render. Tables by area:

Area	Tables
Website	site_content (editable text, photos, documents), live_updates (videos and YouTube links), academic_calendar (weekly PDFs), admission_applications
Store and fees	store_products, store_orders, class_fee_schedules, individual_fee_charges, fee_payment_reports, school_commerce_settings (instructions and bank accounts)
Portal people	portal_classes, portal_students, portal_teachers, portal_sessions, portal_account_tokens, portal_id_sequences, portal_login_events
Teaching	portal_exams, portal_attempts, portal_materials, portal_notices, portal_files (exam PDFs)
Report cards	portal_terms, portal_subjects, portal_scores, portal_report_cards, portal_report_releases
Student life	portal_events, portal_albums, portal_album_photos, portal_images (all private photos, including payment screenshots)
Files and audit	media_objects (website files on Cloudinary), portal_audit_log
Staff accounts	users, sessions (links from Clerk sign-in to the school's records)

Rule for table changes: the build applies table changes to the live database automatically, so changes must only **add** (new tables, new columns with defaults). Never drop or rename a column without a planned migration and a backup.

7 · ACCESS

Sign-in, roles and permissions

Role	How they sign in	What they can do
Website administrator	Clerk, approved email listed in SCHOOL_ADMIN_EMAILS	Everything in /admin and the Learning portal.
Super administrator	As above, listed in SCHOOL_SUPER_ADMIN_EMAILS (or the first admin)	Also sees the audit log.
Teacher	Clerk, email approved in the Learning portal (Teachers)	Only their own classes: exams, marking, notes, notices, photo approvals.
Student	Student ID (e.g. NWE/2026/00001) and their own password	Their own exams, results, report cards and student-life features.
Visitor	No sign-in	Public website, admissions form, store, payment reports.

Student accounts

- The office enrols a student; the site creates the Student ID automatically (NWE/*year*/*number*).
- An email with the Student ID and a one-time **create-password link** (valid 72 hours) goes to the student or parent email.
- After the student chooses a password, a **confirmation email** with their sign-in details follows (never the password).
- Passwords: at least 10 characters with a letter and a number, stored as scrypt hashes. Repeated wrong passwords lock the account for a while.
- Sessions can be listed and revoked by administrators; a password change signs out all other devices.

Staff sign-in

Staff use Clerk with **email verification codes**. Clerk's free plan has no two-factor in production, so **REQUIRE_ADMIN_MFA must stay off**; the admin email inbox is therefore the key to admin access and needs strong protection (strong password and Google 2-Step Verification).

8 · FEATURES

Features by area

Area	Features
Public website	Homepage sections editable in /admin (text, photos, documents, show/hide), admissions form, news, live updates (uploaded video up to 500 MB or YouTube link), weekly calendar PDFs, gallery, prospectus download, top bar with Student Portal and Admin Portal.
Payments and store	Store catalogue with stock, order requests, fee schedules, individual charges, up to three school bank accounts with copy buttons, payment reports, payment screenshots (also “Already paid?” later), office review.
School office (/admin)	Dashboard with key numbers and Learning portal card, admissions with status, website content editor, live updates, academic calendar, graphics studio, store and fees, Media storage test, Email test.
Learning portal (staff)	Classes, students (single and bulk enrolment with activation emails), teachers (role, subjects, photo, directory), CBT exams (interactive or PDF, timers, results release), marking, report cards (terms, subjects, scores, broadsheet, printable cards), notes, notices, events calendar, gallery albums, photo approvals, sessions, audit log.
Student portal	Phone-style home (tiles, bell, search, bottom bar), exams, grade sheet, report cards, library (notes), notices, events calendar, staff directory, gallery, profile with photo (approved by staff) and password change, dark mode.

9 · FILES AND EMAIL

Files, video and email

Cloudinary

- **Website files** (photos, videos, calendar PDFs, product photos) are public and served from Cloudinary's CDN. The site records each file in *media_objects*.
- **Portal files** (exam PDFs, student and staff photos, payment screenshots) are **private** (Cloudinary authenticated) and reach people only through the site's permission checks.
- Uploads use **signed, expiring upload links**; the browser sends files without cookies, so the link itself is the permission.
- Videos over 95 MB upload from the browser straight to Cloudinary in 20 MB pieces, each piece freshly signed. Cloudinary's free plan refuses files over 100 MB; long videos can be YouTube links instead (played through youtube-nocookie.com).
- Everything sits under the **nwe/** folder in the Cloudinary Media Library; deleting or renaming files there breaks the site.

Resend

- Sends from the verified domain (EMAIL_FROM, e.g. New World Emerald <noreply@newworldemeraldprivateschool.com>).
- Emails: student activation (Student ID + create-password link), account ready (sign-in details), password reset, password changed, admin test email.
- Optional: Clerk's staff sign-in emails can be routed through Resend with a signed webhook (/api/webhooks/clerk-email).
- /admin → Email → **Send test email to me** shows Resend's exact answer; /admin → Media storage → **Test connection** does the same for Cloudinary.

10 · SECURITY

Security

Protection	How
Fail-closed admin	Website management addresses are private unless explicitly listed as public.
Request validation	Every request is checked against the generated contract; unexpected fields and sizes are refused.
Headers	HSTS, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy, Content-Security-Policy (report-only until CSP_ENFORCE=true). No X-Powered-By.
Audit log	Staff changes recorded with before/after snapshots and secrets removed; super admins can read it.
Rate limits	Student sign-in attempts, password reset requests and public payment screenshot uploads are limited.
Private files	Student photos, exam PDFs and payment screenshots are never public; each request is permission-checked.
Uploads	File type is checked from the file's own bytes, never the browser's claim; sizes are capped.
Secrets	No keys in the public repository (full history scanned on 24 October 2026: clean). Keys only in Render → Environment.
Dependencies	pnpm audit run; patched versions pinned for Express internals and uuid.

11 · OPERATIONS

Hosting, domain and deployment

- **Code to live site:** push to GitHub → Render builds the deploy branch (currently *chore/hardening-and-tooling*) → *scripts/render-build.sh* checks key formats, installs with pnpm 10, applies table changes, builds → the new version replaces the old one with no downtime.
- **Working branch:** *portal-upgrade*, fast-forwarded into the deploy branch for each release. *main* is behind; see the checklist.
- **Domain:** Cloudflare DNS. @ and *www* are CNAMEs to the Render service; Clerk's five records (clerk, accounts, clkmail, clk._domainkey, clk2._domainkey) and Resend's records are DNS-only.
- **Old addresses:** WordPress paths (/about-us/, /faqs/, /shop/, /my-account/ and theme pages) redirect to their new places; with FORWARD_TO_DOMAIN=true the onrender.com address forwards to the domain.
- **Backups:** Render PostgreSQL paid plans include daily backups with point-in-time recovery; the free plan has none. Manual copies use *pg_dump* (commands in DEPLOY-RENDER.md, section 6).

Never touch the *mail*, *MX* or SPF *TXT* records in Cloudflare, and do not cancel the old hosting plan while school email still runs on it. To undo the domain move, put the old A records back.

12 · SETTINGS

Settings (environment variables)

Set in **Render** → **Environment**. Values are never stored in the code. *.env.example* shows the format.

Variable	Purpose
DATABASE_URL	PostgreSQL connection (Render provides it).
CLERK_PUBLISHABLE_KEY, VITE_CLERK_PUBLISHABLE_KEY	Clerk publishable key (pk_live_…); both must be the same.
CLERK_SECRET_KEY	Clerk secret key (sk_live_…).
CLERK_WEBHOOK_SIGNING_SECRET	Optional: verifies Clerk's email webhook (whsec_…).
SCHOOL_ADMIN_EMAILS / SCHOOL_SUPER_ADMIN_EMAILS	Who may manage the website / read the audit log.
REQUIRE_ADMIN_MFA	Keep false (Clerk free plan has no production two-factor).
CLOUDINARY_URL	cloudinary://API_KEY:API_SECRET@CLOUD_NAME (the only Cloudinary setting the site reads).
RESEND_API_KEY, EMAIL_FROM	Email sending and sender address.
PUBLIC_SITE_URL	https://newworldemeraldprivateschool.com (used in emailed links).
FORWARD_TO_DOMAIN	true: the onrender.com address forwards to the domain.
CSP_ENFORCE	true to enforce the Content-Security-Policy (after reviewing reports).
FRONTEND_DIST_DIR, NODE_ENV, NODE_VERSION	Set by the Render blueprint; do not change.
GCS_SERVICE_ACCOUNT_JSON, PRIVATE_OBJECT_DIR, PUBLIC_OBJECT_SEARCH_PATHS	Only for the Google Cloud Storage alternative; not used with Cloudinary.

13 · QUALITY

Testing and quality

Results of the full run on 24 October 2026:

Check	Result
Type check, every package	Pass
Screen tests (Vitest)	134 tests, 20 files, all pass
API unit tests	56 pass
Upload and file tests	14 pass
Database suites	9 suites pass: portal control, report cards, security, accounts, community, media, media storage, school payments, deploy
Generated code matches contract	Pass (codegen produces no changes)
Homepage size budget	208 kB of 220 kB
Live site crawl (phone and desktop)	All pages 200, no script errors, no broken images, no sideways scrolling
Security	Headers present, HTTPS forced, git history free of secrets
GitHub Actions CI	Runs all of the above on every push; passing

Commands are listed in README.md. Database tests need a throwaway PostgreSQL in DATABASE_URL, **never the live database**.

14 · HISTORY

How it was developed

When	Milestone
2 to 3 Oct 2026	Website and backend first built on Replit: public site, admin panel, admissions, store and fees, email sign-in.
23 Oct 2026	Moved to a Windows workflow; tooling hardened, CI added, Render deployment prepared; student portal with live exams; report cards; premium redesign of staff tools and student portal.
23 to 24 Oct	Phase 1 security (audit log, super admin, sessions, headers); Phase 2 student accounts (automatic IDs, activation links).
24 Oct	Student portal: phone-style home, events calendar, staff directory, gallery, profile photos; /admin redesigned into the app shell.
24 Oct	Full site check; homepage spacing, palette and layout polish; Cloudinary media storage; clean build logs.
24 Oct	Domain moved from WordPress to this site; Clerk production; Resend email with confirmations; YouTube and 500 MB videos.
24 Oct	Handover: README, CI on live branches, prospectus, logo and scroll fixes, bank accounts and payment screenshots, this document.

Method used throughout: explore, plan, agree each phase, build with tests, check in a real browser, commit on a branch, deploy and verify on the live site. The full history (38+ commits) is in GitHub.

15 · HANDOVER

Handover checklist

Transfer ownership

Account	Hand over	Done
GitHub repository	Add the client as owner, or transfer the repository	
Render (web service + database)	Invite the client to the team, then make them owner	
Cloudflare (DNS)	Add the client as Super Administrator	
Domain registrar	Confirm the client owns the domain registration	
Clerk (production instance)	Transfer the application to the client's organisation	
Cloudinary	Invite the client as admin, or transfer the account	
Resend	Invite the client to the team	
Old hosting (school email)	Confirm the client controls it; keep it paid while email runs there	

Security tidy-up

Action	Done
Rotate every key that was shared during development (database password, Clerk keys, Cloudinary secret, Resend key) and update Render	
Delete unused Render variables (CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET, CLOUDINARY_CLOUD_NAME)	
Confirm REQUIRE_ADMIN_MFA is false or removed	
Set SCHOOL_ADMIN_EMAILS / SCHOOL_SUPER_ADMIN_EMAILS to the client's staff and remove the developer	
Remove or reset test accounts (for example test students and teachers)	
Turn on Google 2-Step Verification for every admin email account	
Merge portal-upgrade into main and point Render at main	

16 · NEXT STEPS

Known items and recommended next steps

Item	Status and recommendation
Zod security advisory	One audit finding needs a Zod 4 migration, which breaks generated types today. Risk is low (request bodies are capped at 2 MB, lists have size limits). Plan a dedicated upgrade.
Content-Security-Policy	Runs in report-only mode. Review /api/csp-report entries, then set CSP_ENFORCE=true.
Database backups	Upgrade Render PostgreSQL to a paid plan for daily backups, or schedule manual pg_dump copies.
Cloudinary plan	Free plan: videos up to 100 MB. Upgrade for 500 MB uploads, or keep using YouTube links.
Clerk plan	Upgrade if staff two-factor is wanted, then set REQUIRE_ADMIN_MFA=true.
Planned features	Attendance and assignments, chat with class teacher and office, notifications, fees per student, library upgrade, CBT question bank and lockdown, import tool, admin and staff user guides.
Old full portal test	test:portal relies on Replit object storage and is not run on Render; the database suites cover the same areas.

Where to start as a new developer: read README.md, then docs/DEPLOY-RENDER.md, run the checks, and make every API change contract-first in lib/api-spec. Keep keys out of the code, keep table changes additive, and never touch the school email DNS records.